EECS 484 - Database Management Systems

# Tree-Structured Indexes

Chapter 10

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

# Index Design Space

## Organization Structure for k*

- Hash-based

  (+) Equality search
- Tree-based

  (+) Range, equality search
  - B+Tree (dynamic)
  - LSM Tree (dynamic)
  - ISAM (static)

## Data Entry (k*) Contents

1. Actual Data record

   index = file
2. <k, rid>

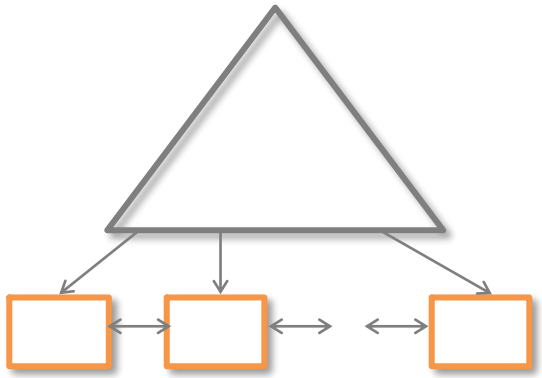
   actual records in a diff file
3. <k, list of rids>

# Motivation

- Range and equality searches very common
- Can scan heap file, but this is expensive
- Goal: Create a dynamic index structure that allows for efficient evaluation, and equality / range queries.

# B+ Tree

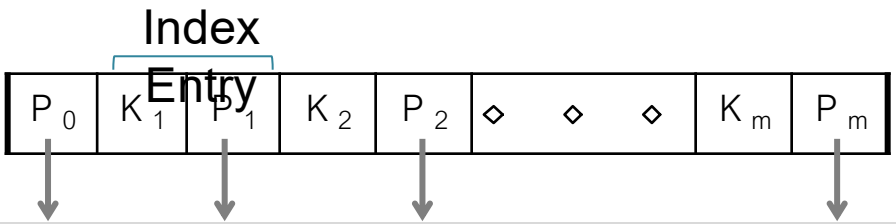
- Height-balanced (dynamic) tree search structure
- Supports equality and range-searches efficiently
- Main Characteristics:
  1. Minimum 50% occupancy (except for root), i.e. each node contains at least d entries, where d = order of the tree, i.e., d=ceiling(max number of entries / 2).
  2. Height of tree: the length of any path from the root to a leaf.
  3. Remains balanced after insert/delete operation.



Index entries
(direct search)

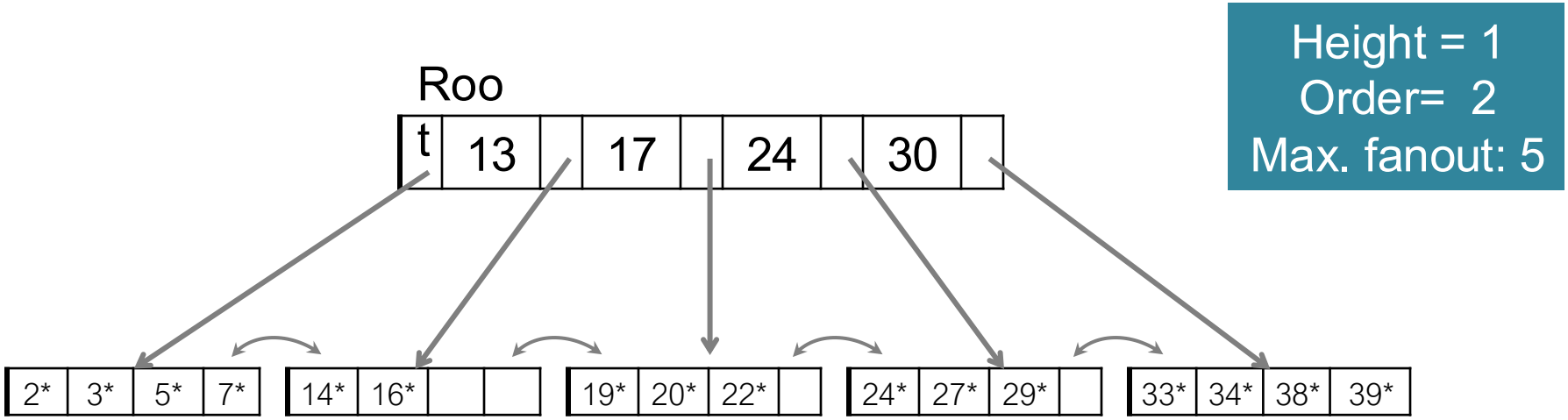
Data entries

Index Entries:
Entries in the non-leaf pages:
(search key value, pageid)

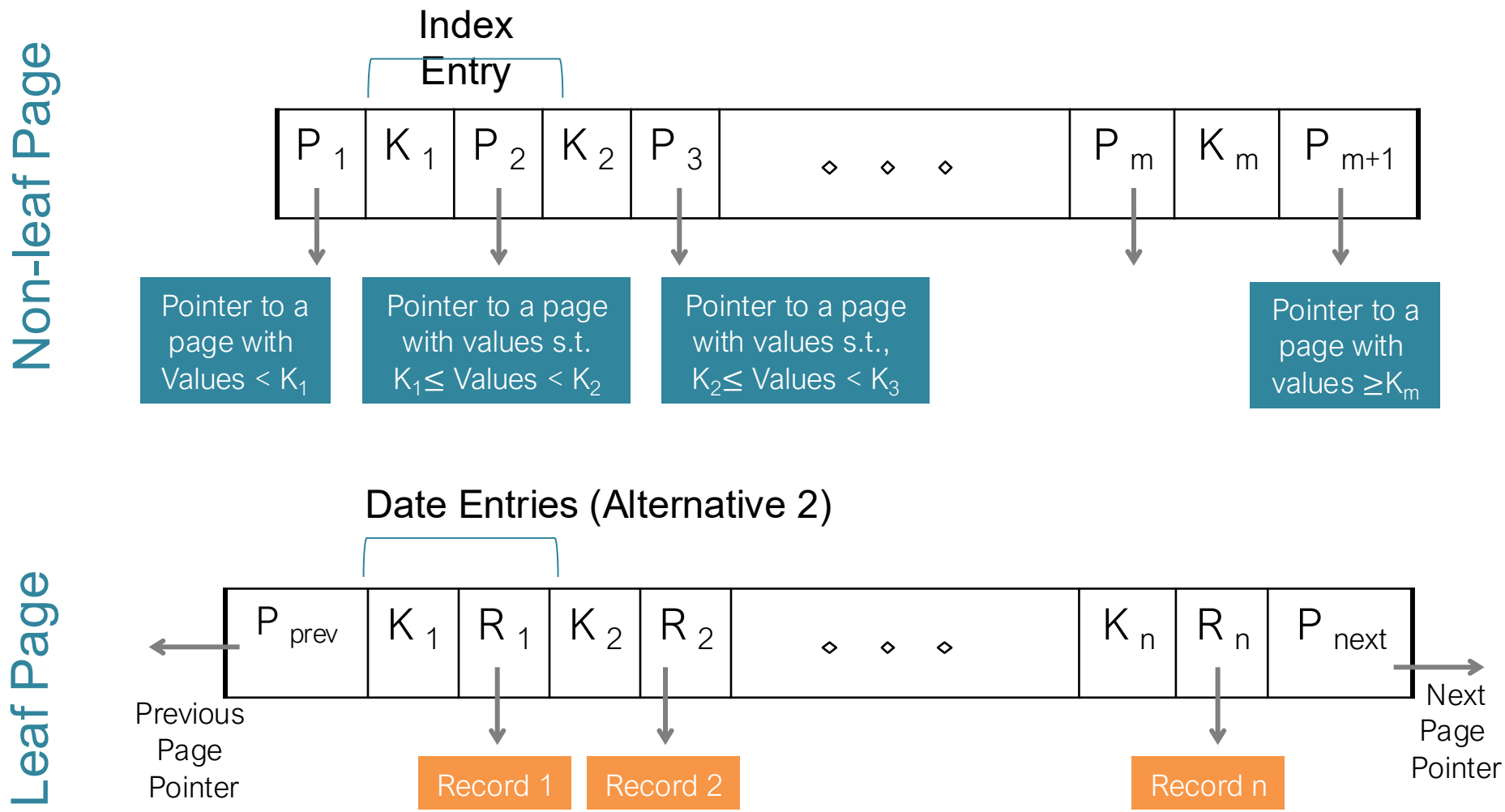


# Searching in a B+ Tree

- Starting from root, examine index entries in non-leaf nodes, and traverse down the tree until a leaf node is reached
  - Non-leaf nodes can be searched using a binary or a linear search.
- Try it out: Search for 5*, 15*, all data entries >=24*



# B+-Tree Page Format



# B+ Trees: Height

What is the height of a B+ tree? (formula)

**Definitions:**

F: fanout (average number of children for non-leaf node)

N: total leaf pages

$\log_F N$

# B+ Trees in Practice

- Typical order 100. Typical fill-factor 66.5%
  - Maximum fanout: 201
  - Average fanout = 133
- Typical capacity:
  - Height = 1: 133 pages of data entries (leaf pages)
  - Height = 2: $133^2$ pages of data entries
  - Height = 3: $133^3$ (> 2 million) pages of data entries
  - Height = 4: $133^4$ (> 300 million) pages of data entries
- Can often keep top levels of index in buffer pool
  - Level 1 = 1 page = 8 Kbytes
  - Level 2 = 133 pages = 1 Mbyte
  - Level 3 = 17,689 pages = 133 MBytes

# Quiz

- You are given a file of 10 million records
- Suppose you can store 10 data entries per leaf page
- You build a B+ Tree with order 75, 67% average fill-factor

- What is the avg fanout?
- What is the height of your B+ Tree?

# Anwer

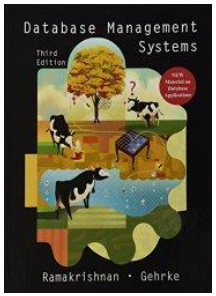
- You are given a file of 10 million records
- Suppose you can store 10 data entries per leaf page
- You build a B+ Tree with order 75, 67% average fill-factor

- What is the avg fanout?
- What is the height of your B+ Tree?
  - Avg fanout = 100 --- 67% * (2d+1) ~= 100, where d = 75.
  - Pages needed for 10M records? 1M pages
  - Height of the B+ tree: logF(N) = log_100 (1M pages) = 3

# A Note on Order

- Some literature uses: order = max # of entries
- **In this class**: order = minimum # of entries

  (book uses this)




- *Order d* concept replaced by physical space criterion in practice (e.g. at least half-full)
  - Index (i.e. non-leaf) pages can typically hold many more entries than leaf pages because they only keep pointers and entries not the data entries

# B+ Tree Operations

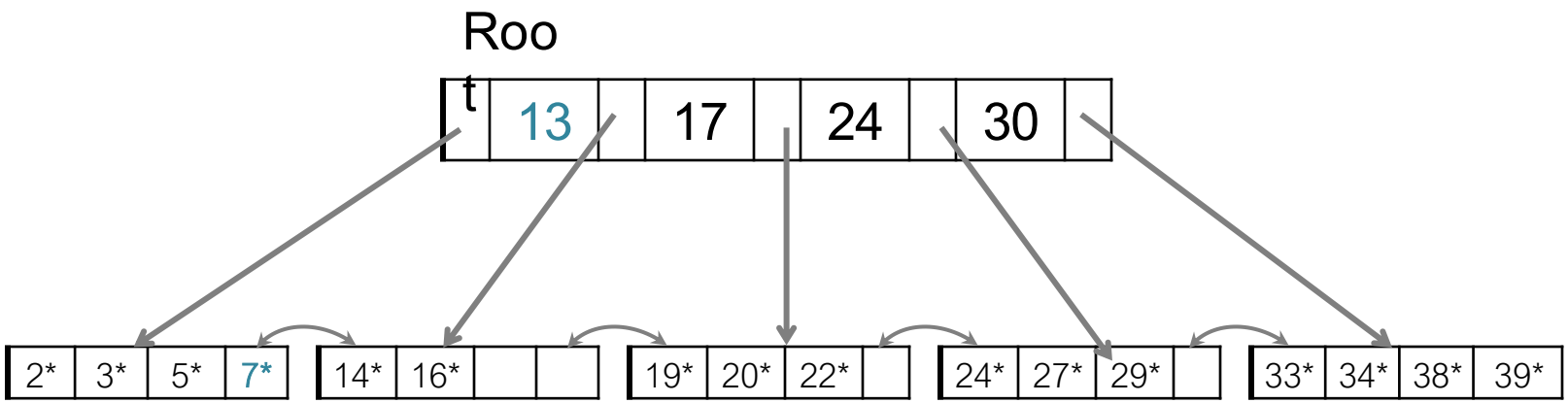
- Search
  - Equality
  - Range
- Insert data entry
- Delete data entry
- Bulk load

# B+ Tree: Inserting a Data Entry

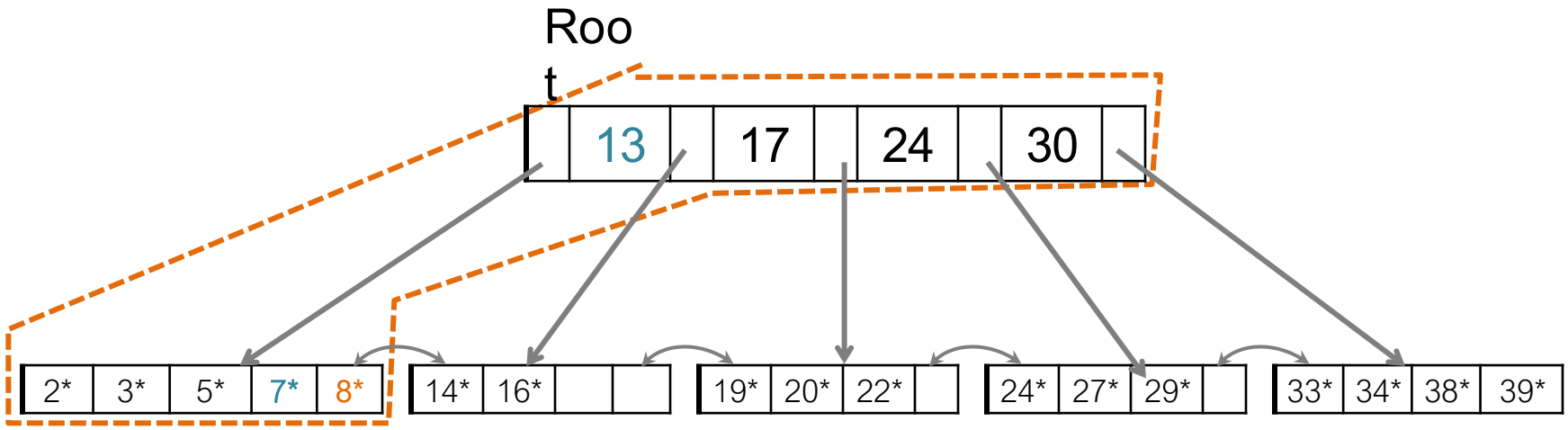
- Maintain invariants:
  - Search-tree property
  - All nodes must be at least ½ full (except root node), i.e., has between d and 2d entries
  - Root node is allowed to have a single entry

- Strategy:
  - Split nodes when they become full and a node is added:
    - Split an overfull node with (2d + 1) entries into two nodes with d and (d+1) entries, resp.

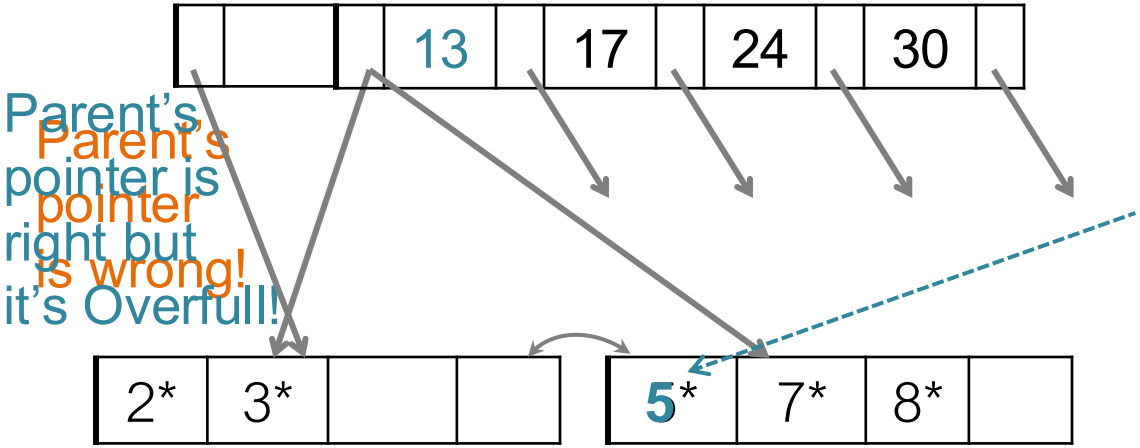
# Inserting 8* into B+ Tree



# Inserting 8* into B+ Tree



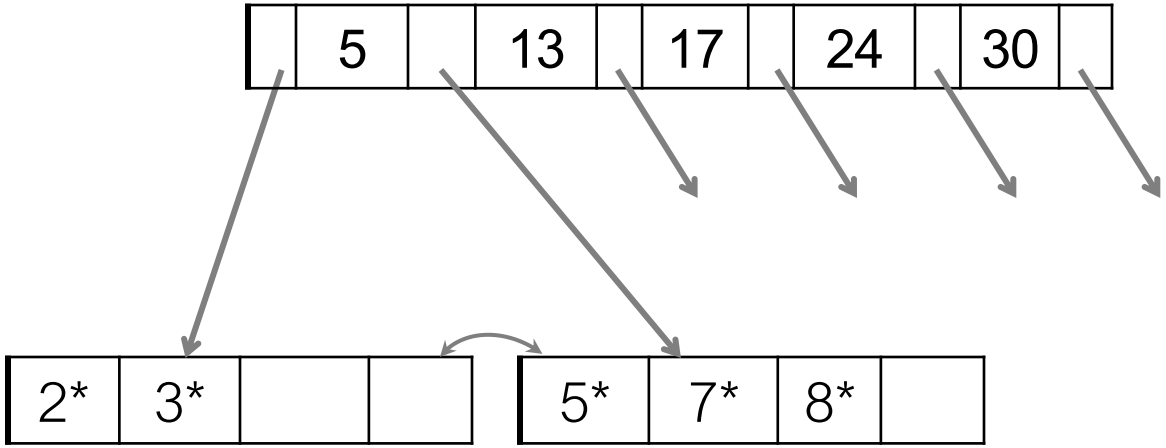

Overfull




Leaf split: The middle key is copied up to the parent (and continues to appear in the leaf)

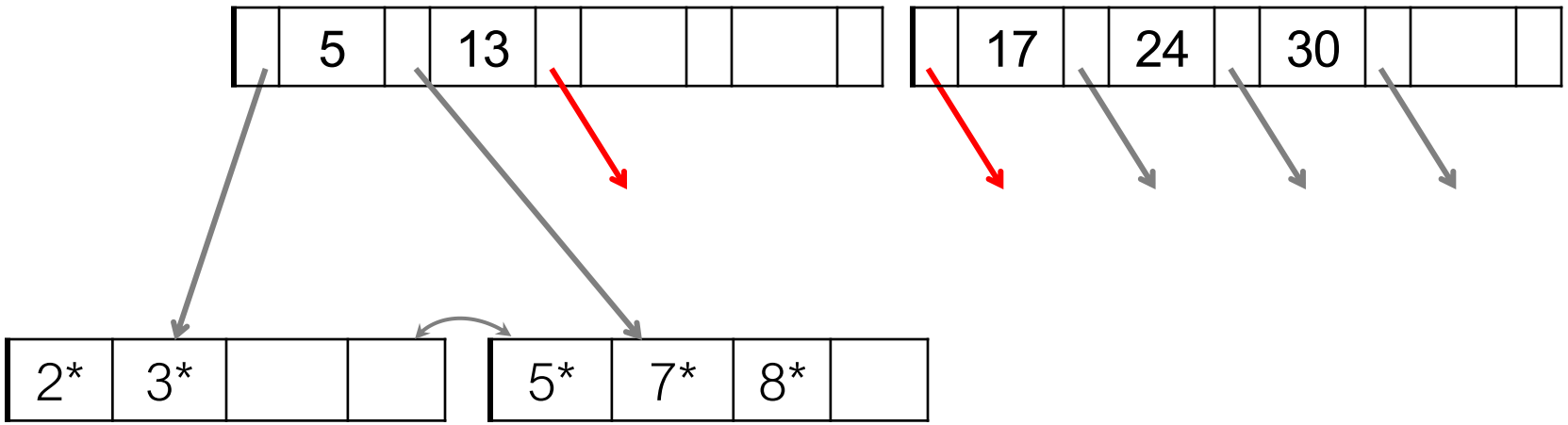
# Splitting Overfull Non-Leaf Nodes

Overfull

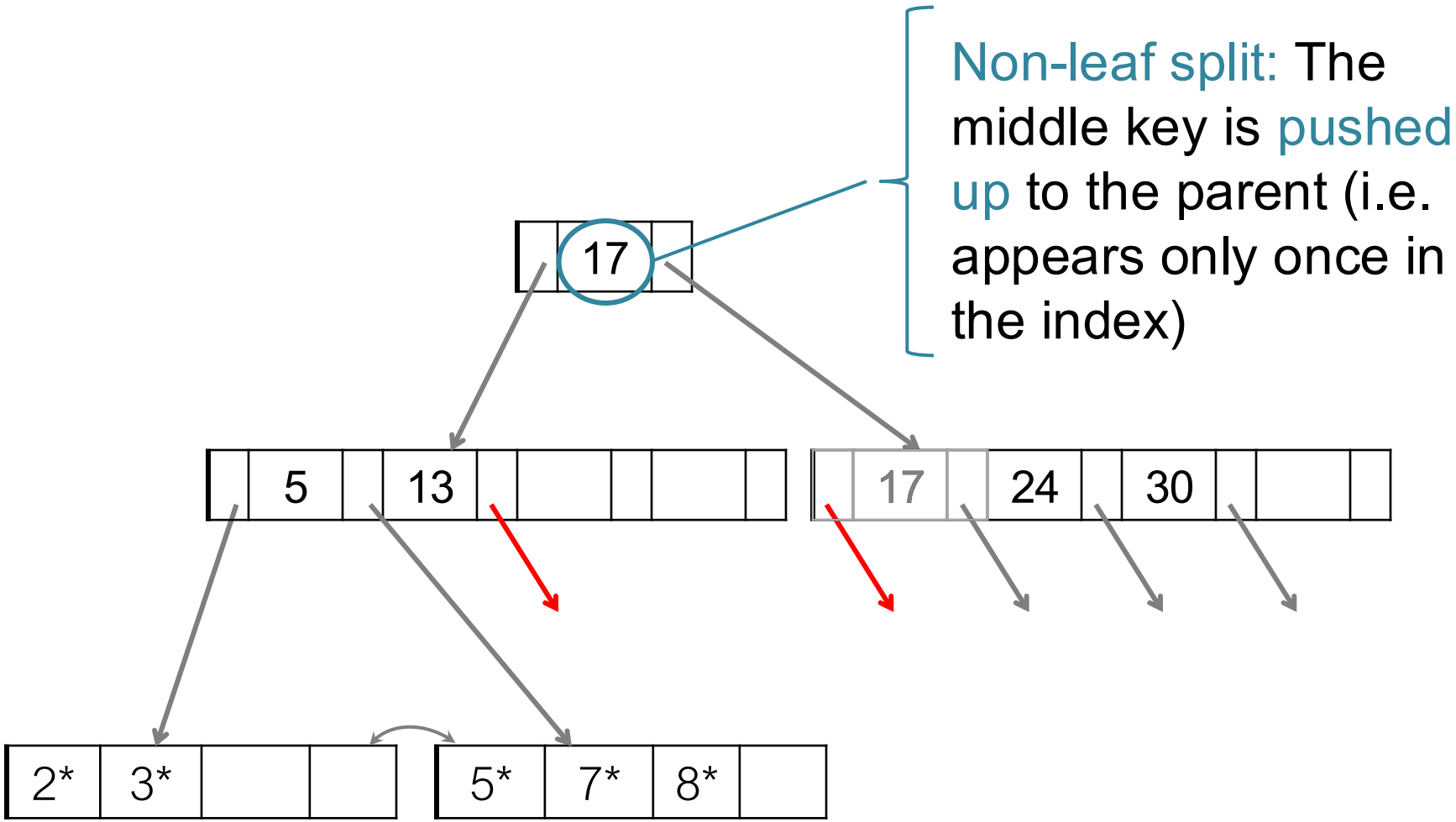


# Splitting Overfull Non-Leaf Nodes

Split

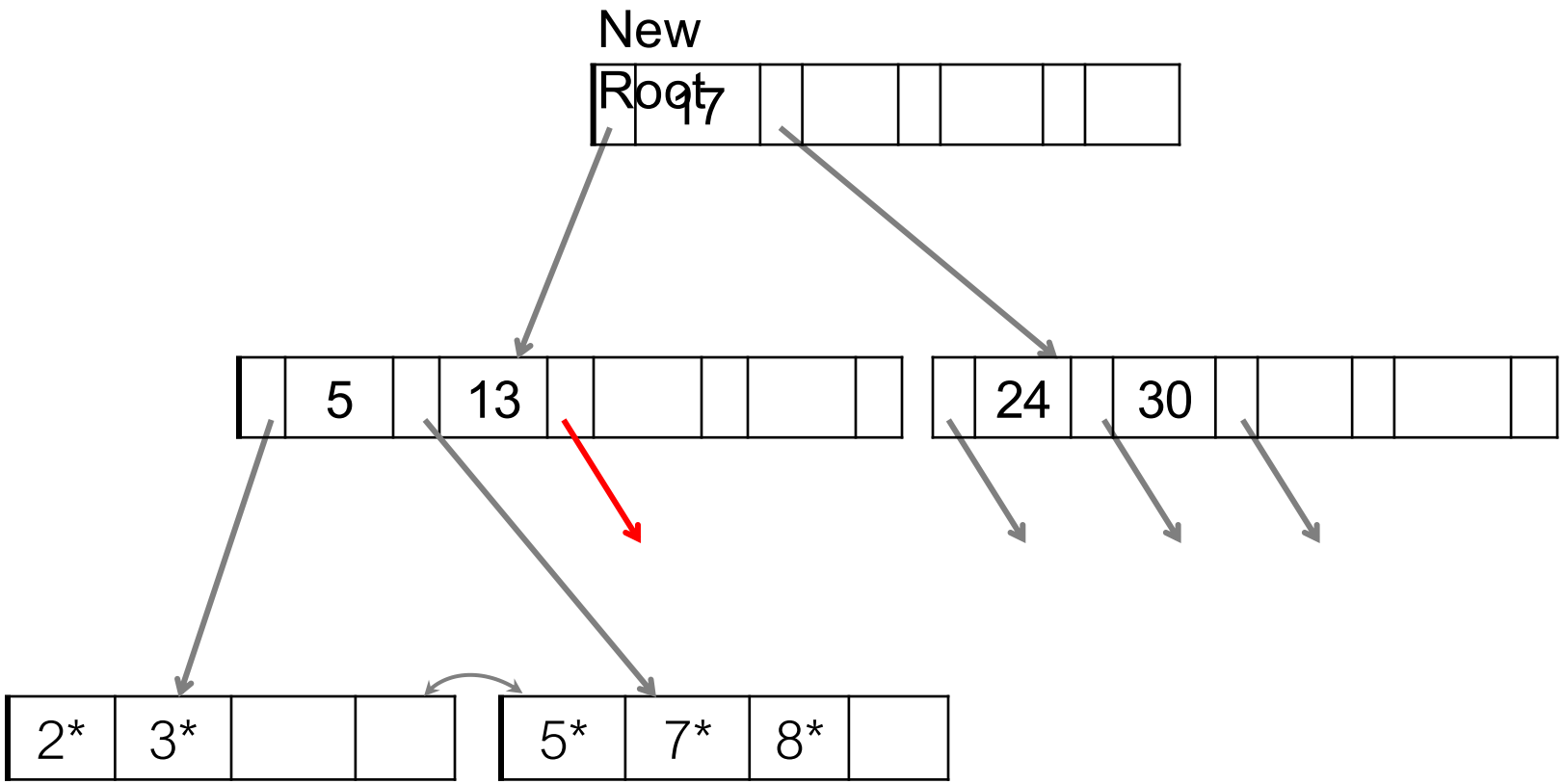


# Splitting Overfull Non-Leaf Nodes

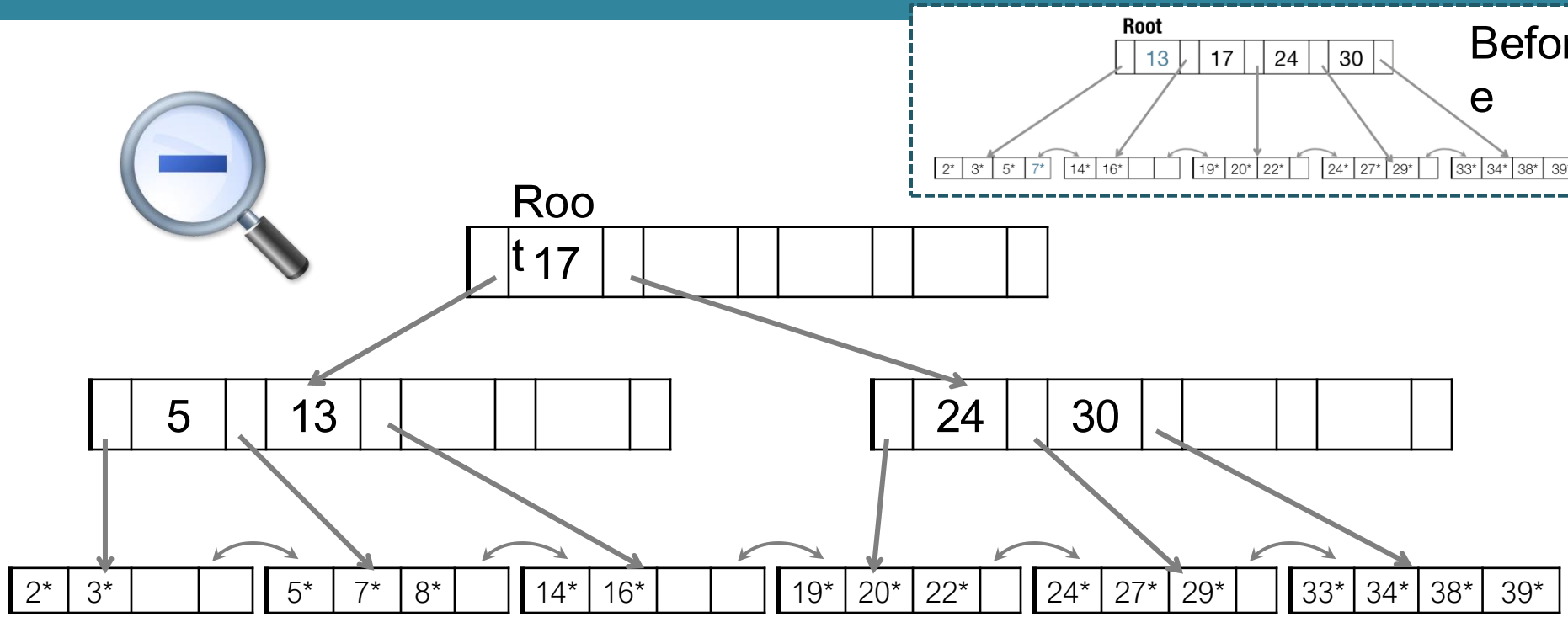


# Splitting Overfull Non-Leaf Nodes

Minimum occupancy is guaranteed in both leaf and index page splits (but not in the root)

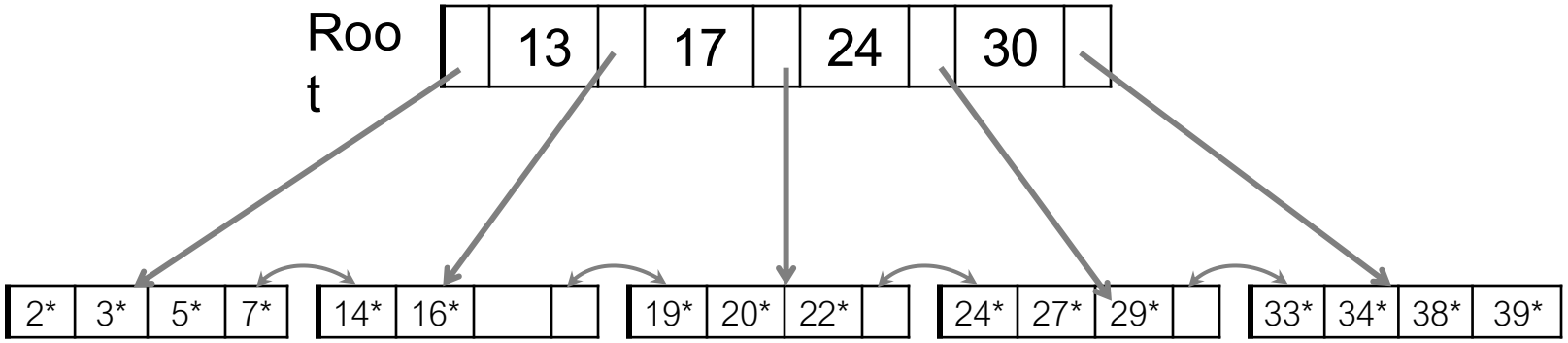


# The B⁺-tree After Inserting 8*

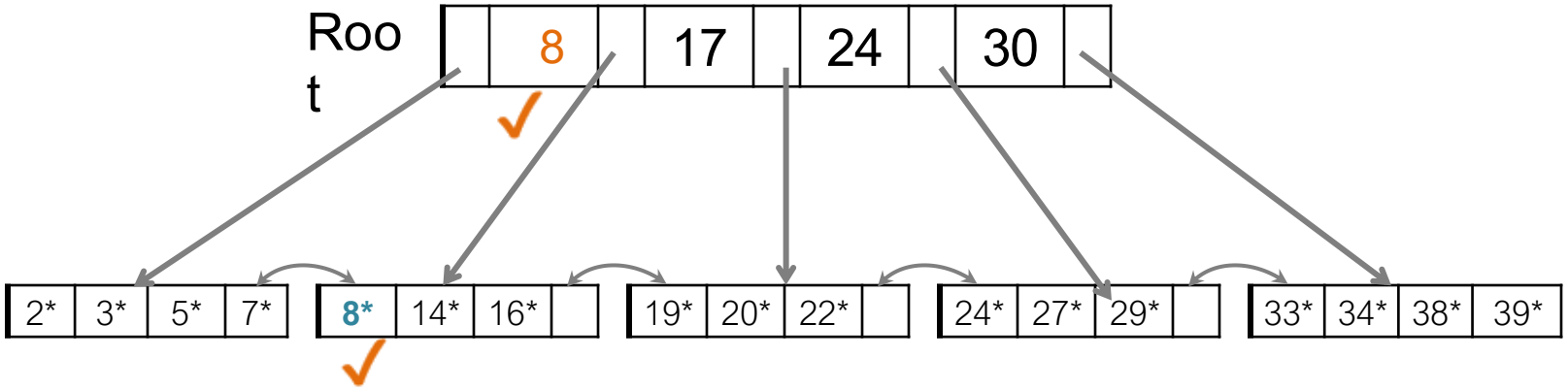



- Root was split: height increases by 1
- Could avoid split by re-distributing entries with a sibling
  - Sibling: immediately to left or right AND same parent

# Inserting 8* via Entry Re-distribution with Siblings



# Inserting 8* via Entry Re-distribution with Siblings




- Re-distributing entries with a sibling
  - + Improves page occupancy, possibly reduces height
  - + May reduce I/O if split would propagate up the tree
  - - Increases I/O (since we check both siblings)
  - Usually not used for non-leaf node splits (cons outweigh pros)
  - Use only for leaf level entries (have to set pointers)
    - No redistribution if next/prev node is full or has diff parent

# Reminder: B+-Tree Insertions

- B+-Tree
  - With order d, each node has up to 2*d entries and 2*d+1 pointers (children)
  - Pointer to left of an entry < and pointer to right >=
  - Minimum 50% occupancy (except for root)
  - Height of tree: length of the path from root to a leaf
  - Remains balanced after insert/delete operation.
- Insertion: Find the right leaf
  1. If full, see if (adjacent) sibling has room, redistribute
  2. Otherwise, split the overfull node with (2d + 1) entries into two nodes with d and (d+1) entries, resp.
  3. May have to split parents recursively

# B+ Tree Operations

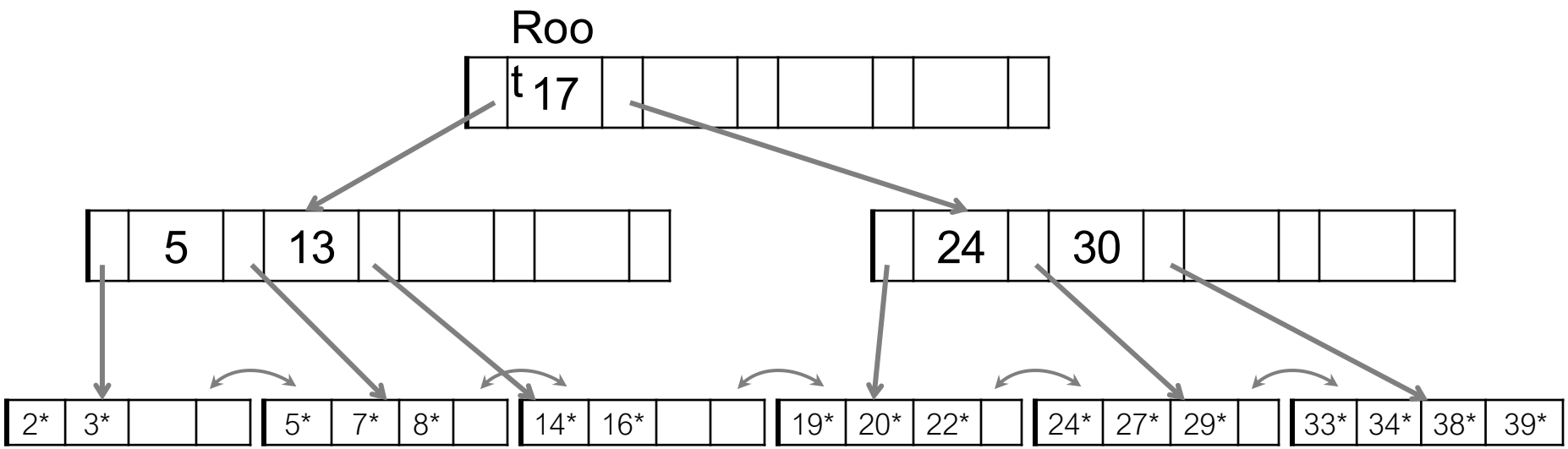
- Search
  - Equality
  - Range
- Insert data entry
- Delete data entry
- Bulk load

# B+-Tree: Deleting a Data Entry

1. Find the data entry (will always be at a leaf)
2. Delete it
3. Restore the B+ tree invariant
   - If L is at least half-full, *done!*
   - If L has only d-1 entries,
     - Try to re-distribute, borrowing from a sibling (adjacent node with same parent as L)
     - If re-distribution fails, merge *L* and sibling
4. On merge, delete relevant entry in parent

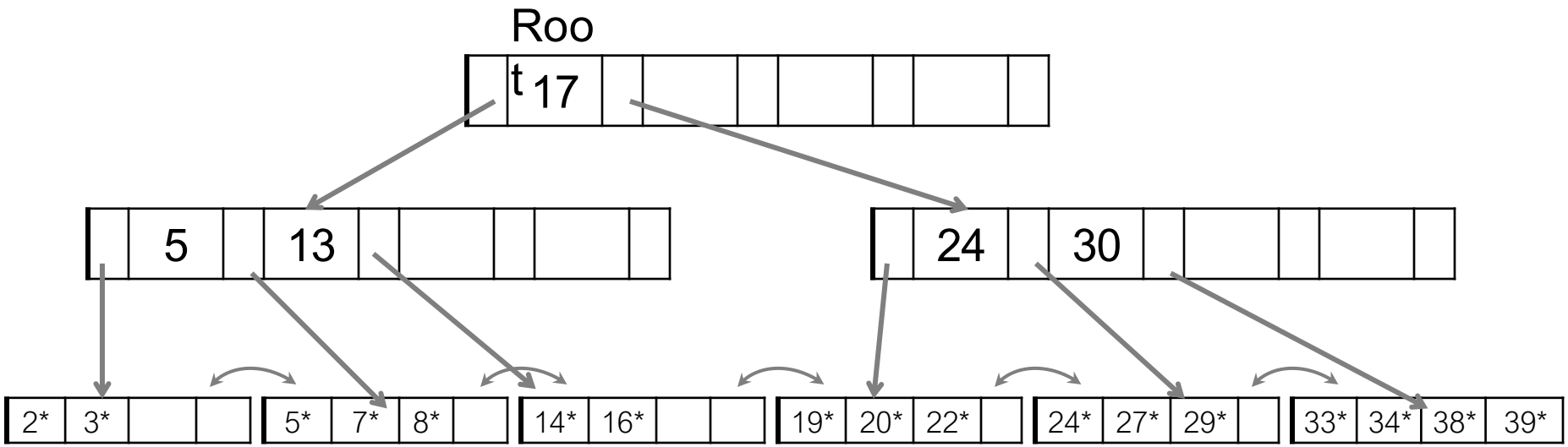
Merge could propagate to root, decreasing height.

# Example Tree



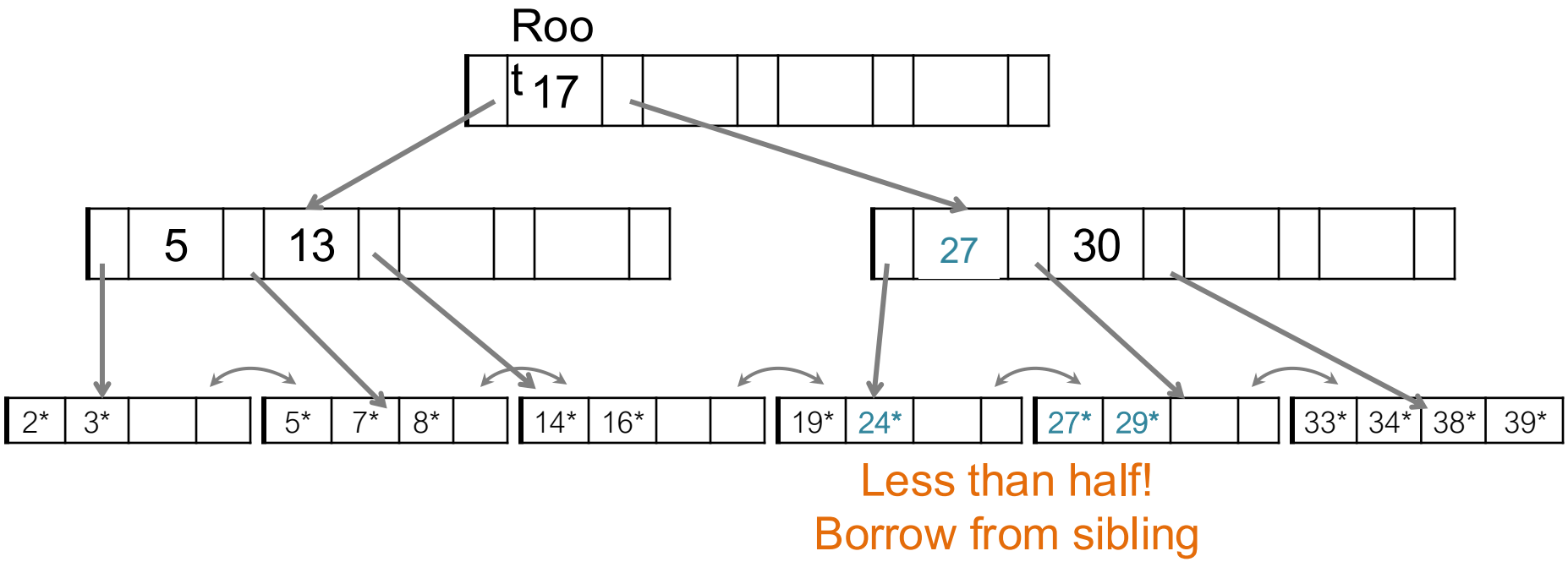

- **Task**: Delete 22, 20, 24
- Deleting 22 is easy. Invariant maintained.
- Deleting 20 is harder. Node would become less than half full.

# Deleting 22* and 20*




Less than half!
Borrow from sibling

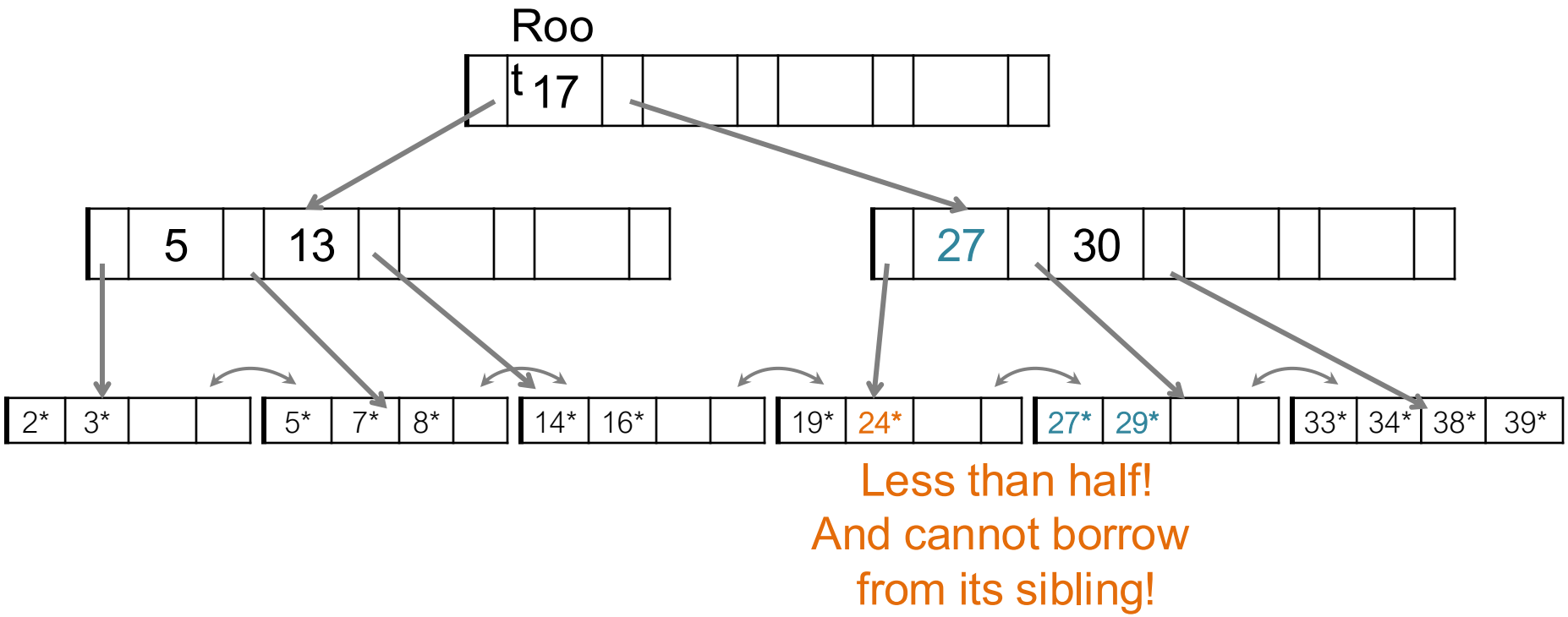
# Deleting 22* and 20*




Deleting 20* is done with re-distribution.

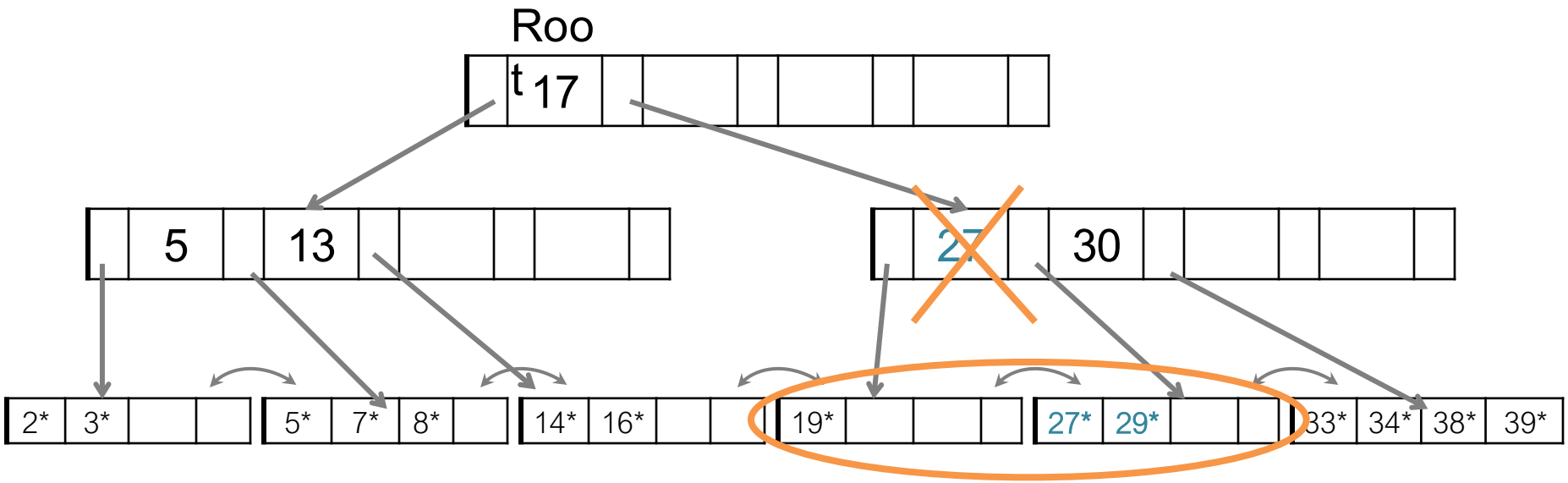
Notice how middle key is copied up.

# ... And then Deleting 24*



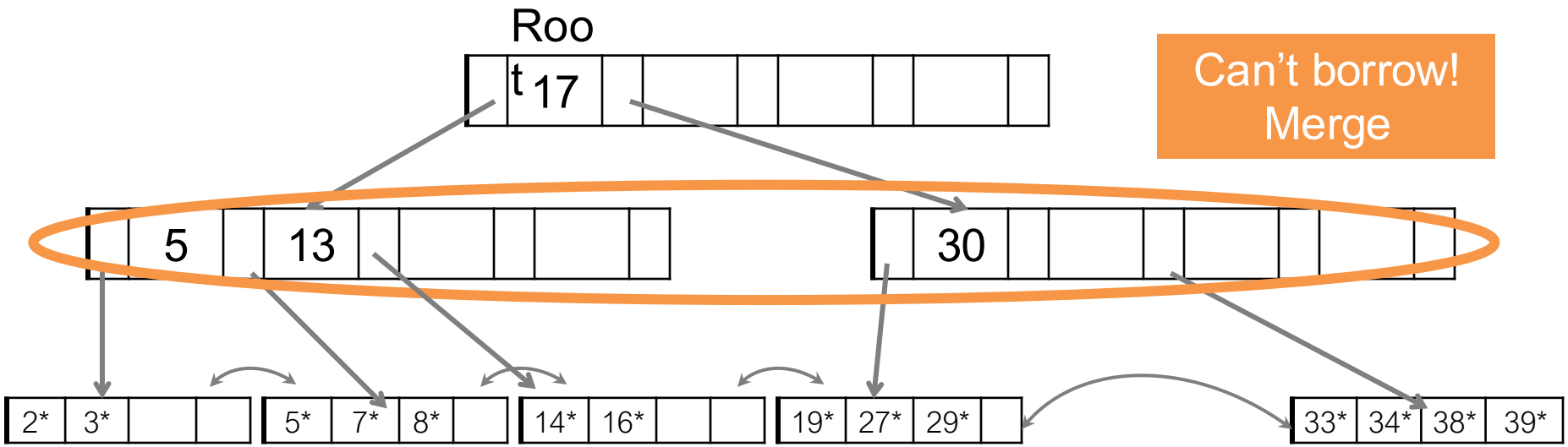

- Must merge
- In the non-leaf node,
  toss the index entry with key value = 27

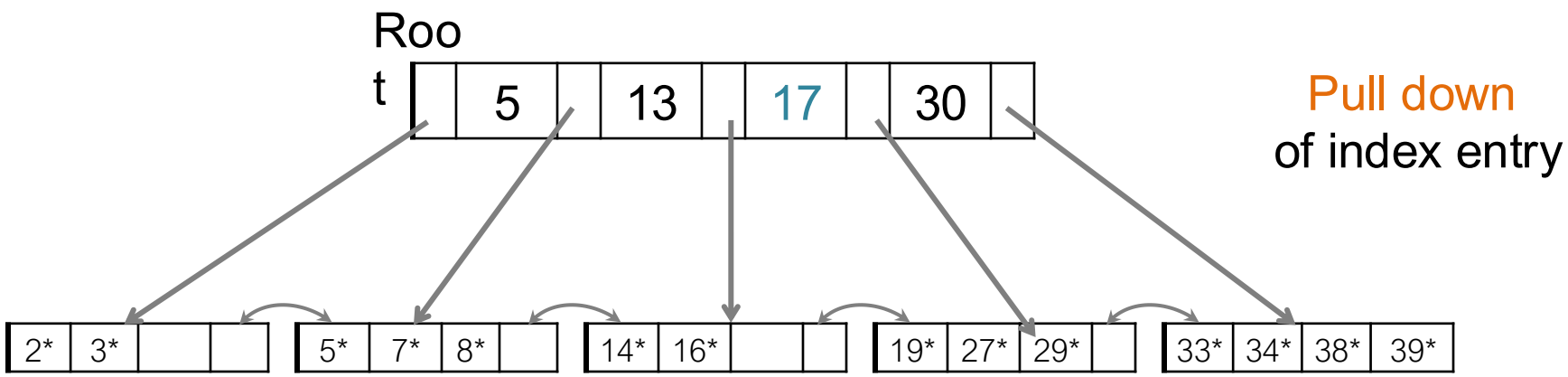
# ... And then Deleting 24*




- Must merge
- In the non-leaf node,
  toss the index entry with key value = 27

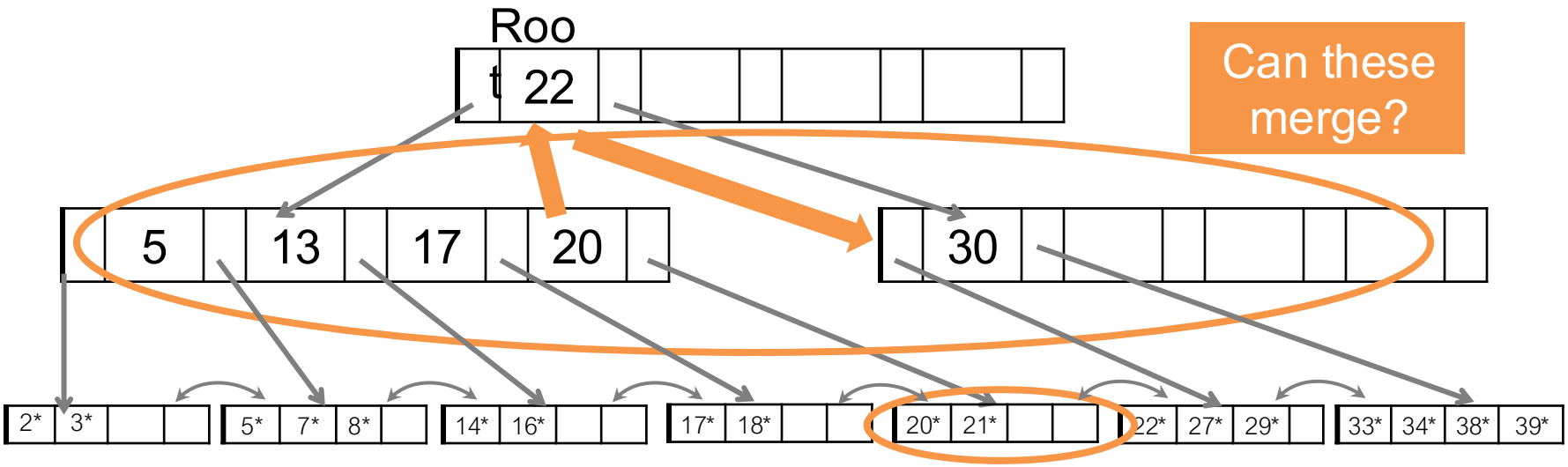
# ... And then Deleting 24*



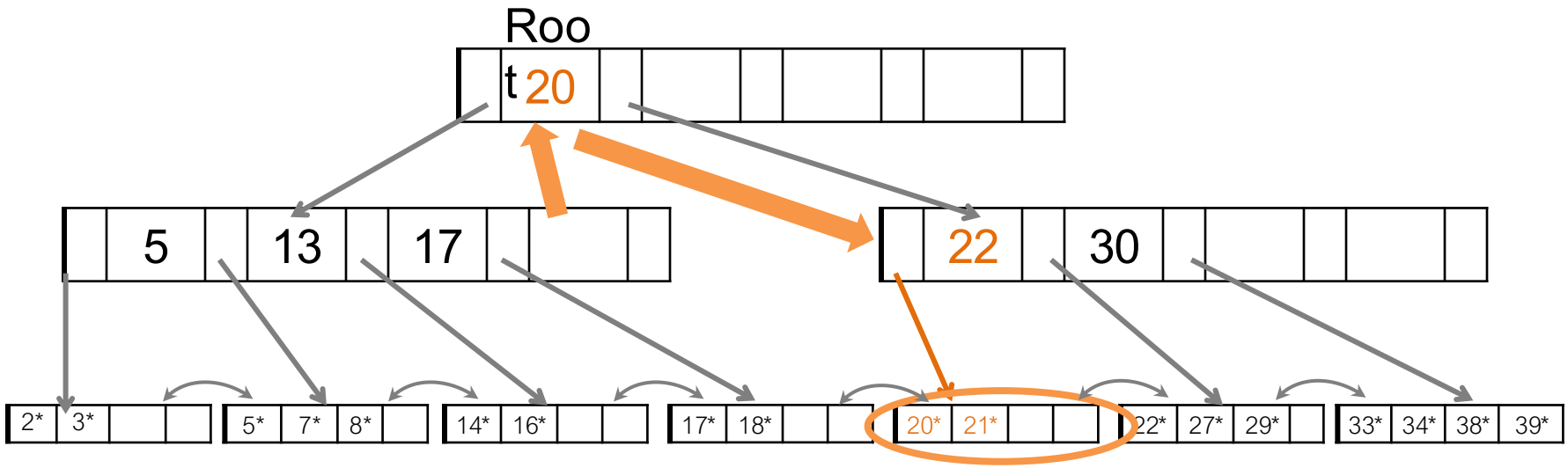
# Another Deletion Example: Non-leaf Re-distribution

Can re-distribute entry from left child of root to right child.



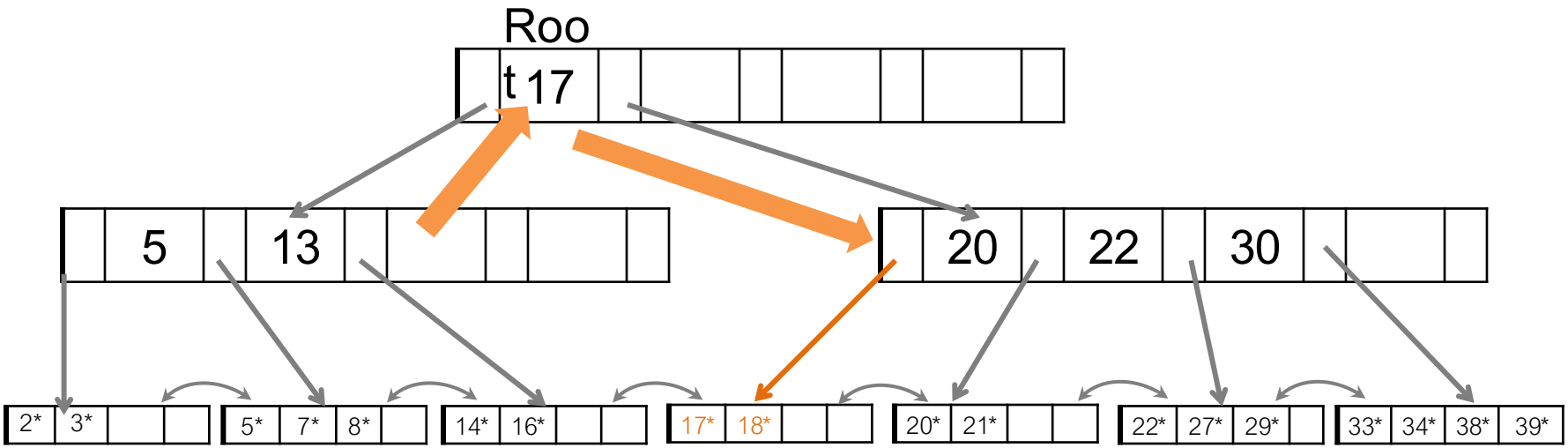
# After Re-distribution

- Rotate through the parent node
- re-distribute index entry with key 20



# After Re-distribution

It suffices to re-distribute index entry with key 20;
For illustration, here we also re-distribute 17



# B+ Tree Deletion

- Try redistribution with all siblings first, then merge. Why?
  - Good chance that redistribution is possible (large fanout!)
  - Only need to propagate changes to parent node
  - Files typically grow, not shrink!

# B+ Tree Operations

- Search
  - Equality
  - Range
- Insert data entry
- Delete data entry
- Bulk load

# Adding Entries to a B+ tree

- Option 1: multiple inserts
  - Slow. Repeated re-organization
  - Does not give sequential storage of leaves
- Option 2: Bulk Loading
  - Fewer I/Os during build
  - Leaves will be stored sequentially (and linked, of course)
  - Can control “fill factor” on pages

# Step 1: Sort Entries on the Search Key

Unsorted input:

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 4 | 17 | 2 | 25 | 11 | 8 | 29 | 14 | 6 | 21 | 3 | 19 | 10 | 27 | 7 | 23 |

↓ *External Sort* ↓

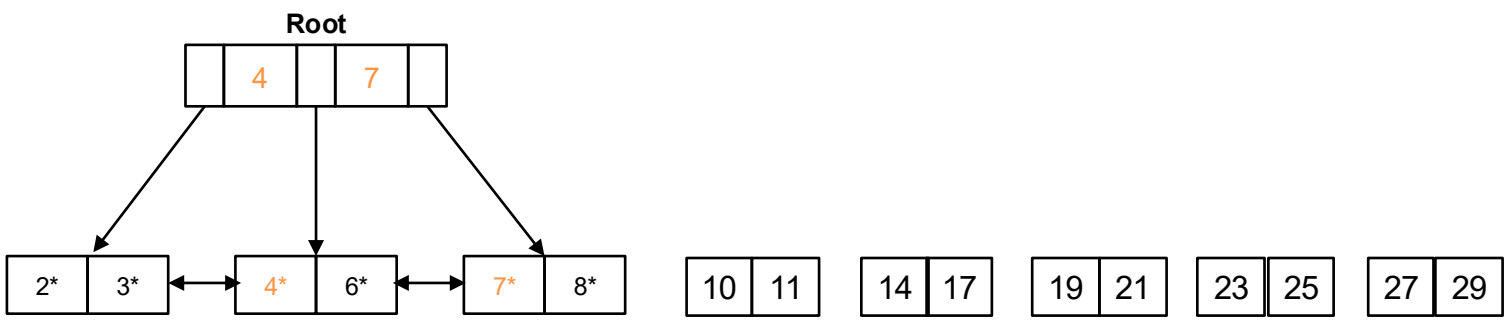
Sorted output:

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 3 | 4 | 6 | 7 | 8 | 10 | 11 | 14 | 17 | 19 | 21 | 23 | 25 | 27 | 29 |

# Step 2: Insert Sorted Entries into Rightmost Leaf

- Keep filling the rightmost leaf.
- When it overflows, create a new leaf and copy up its first key to the rightmost parent

**After inserting 2, 3, 4, 6, 7, 8:**

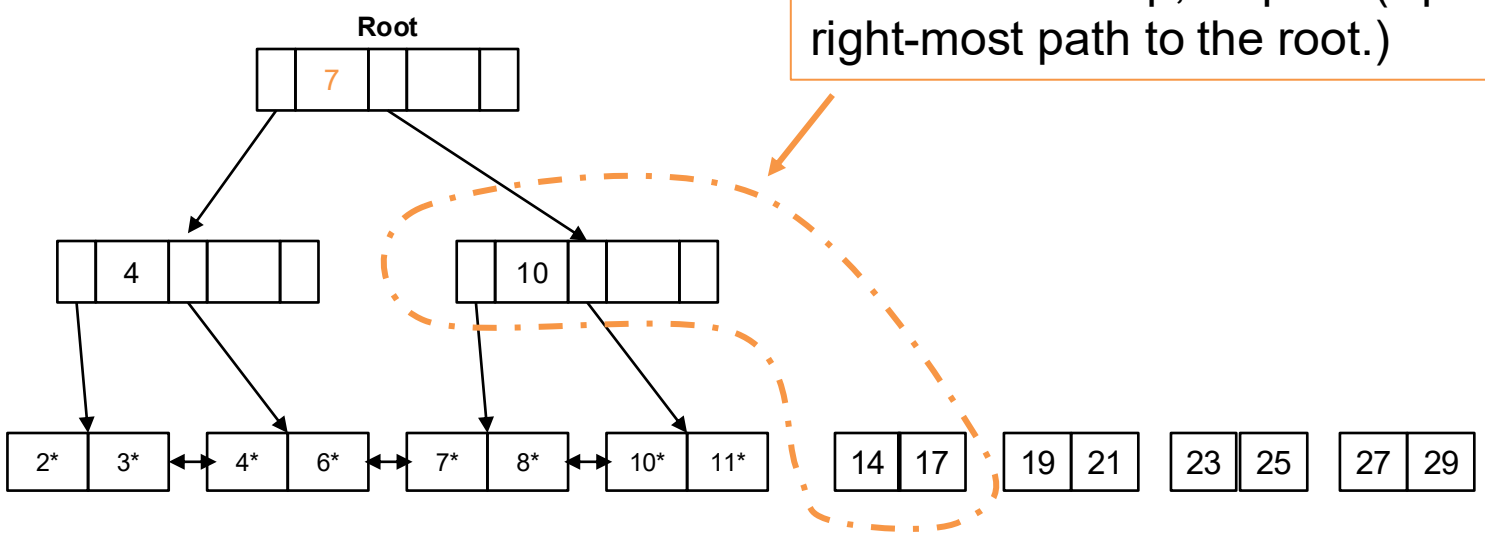


# Step 3: Node Overflow → Split

- If that parent also overflows, split it and push up the middle key.

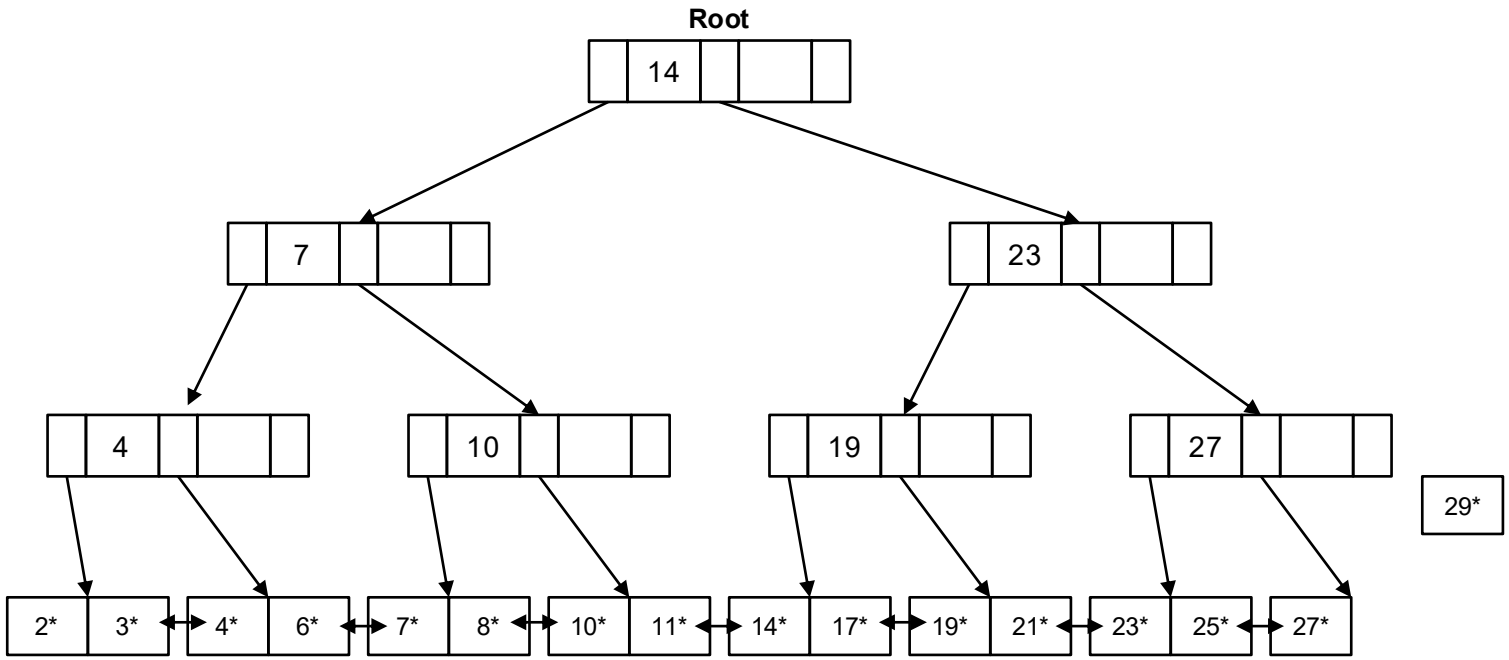
**After inserting 10,11:**

New index entries for leaf pages always entered into right-most index page just above leaf level.
When this fills up, it splits. (Split may go up right-most path to the root.)



# Step 4: Cascading Split→ Final Tree

Inserting 27: [19, 23] overflows → split at 23, push 23 to root.
Root [7, 14] overflows → split at 14.  New root = [14].



# Pseudocode: Bulk Loading

```
BULK_LOAD(entries):

  // Phase 1: Sort
  sorted ← EXTERNAL_SORT(entries)

  // Phase 2: Build tree by always inserting at the rightmost path
  Create first leaf L, insert first entries into L
  Set L as root

  FOR EACH remaining entry e in sorted order:
     IF rightmost leaf is full:
        Create new leaf L'
        Set prev leaf's next-pointer → L'    // sibling pointer
        Copy up first key of L' into rightmost parent
        IF rightmost parent overflows:
           Split it: push up middle key to its parent
           Repeat upward if that parent also overflows
           IF root overflows, split root → new root

     Insert e into the (new) rightmost leaf

  // Key insight: since data is sorted, every new entry always goes
  // to the rightmost leaf — no search from root needed!
```

Optional

# Bulk Loading into an Existing B+ Tree

- Bulk loading works best when building from scratch. If the B+ tree already has data, our options depend on whether the range of new keys overlap with existing ones or not:

## If their key ranges do NOT overlap:

- **Append:** If all new keys are greater (or smaller) than existing keys, bulk-load a new subtree and attach it as the rightmost (or leftmost) child. No existing pages need to be modified.

## If their key ranges do overlap:

- **Option 1: Full rebuild:** Read all existing entries, merge with the new ones, and bulk-load the entire tree from scratch. Worthwhile when the new batch is a large fraction of total data.

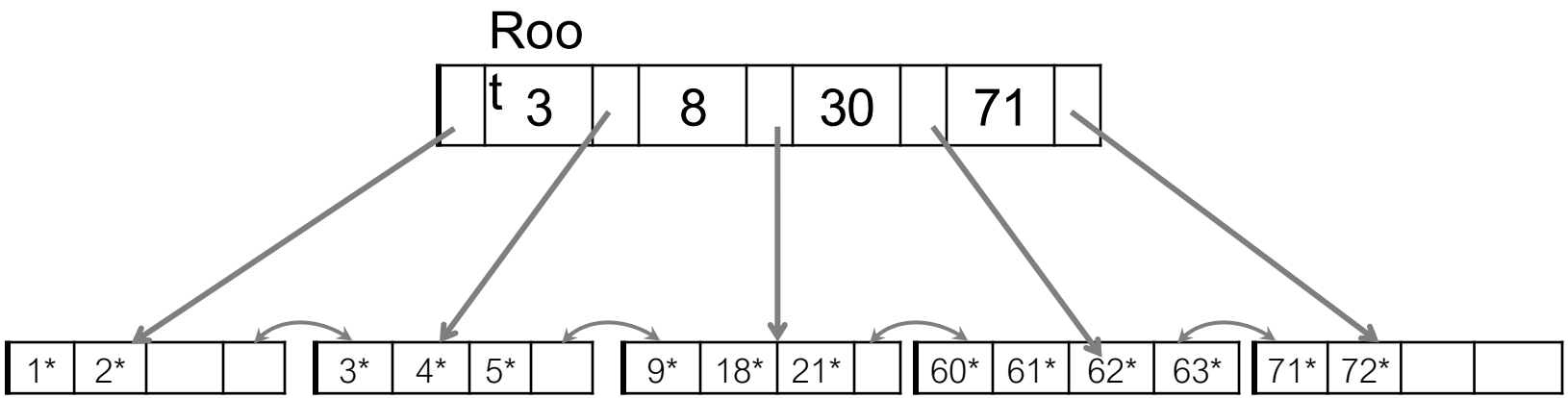
- **Option 2: Hybrid:** Sort the new entries, partition them into ranges matching existing subtrees, and bulk-insert each partition into its corresponding range.

# Summary

- Tree-structured indexes are ideal for range searches, also good for equality searches.
- B+ tree is a dynamic height-balanced index structure.
  - Insertions/deletions/search costs $O(\log_F N)$
  - Split: for leaf (copy up), for internal node (push up)
  - Merge: for leaf (toss), for internal node (pull down)
  - High fanout (F) means depth rarely more than 3 or 4.
  - Max occupancy = 2d, Min occupancy = order = d (excp. root)
  - Most widely used index in database management systems because of its versatility. One of the most optimized components of a DBMS.

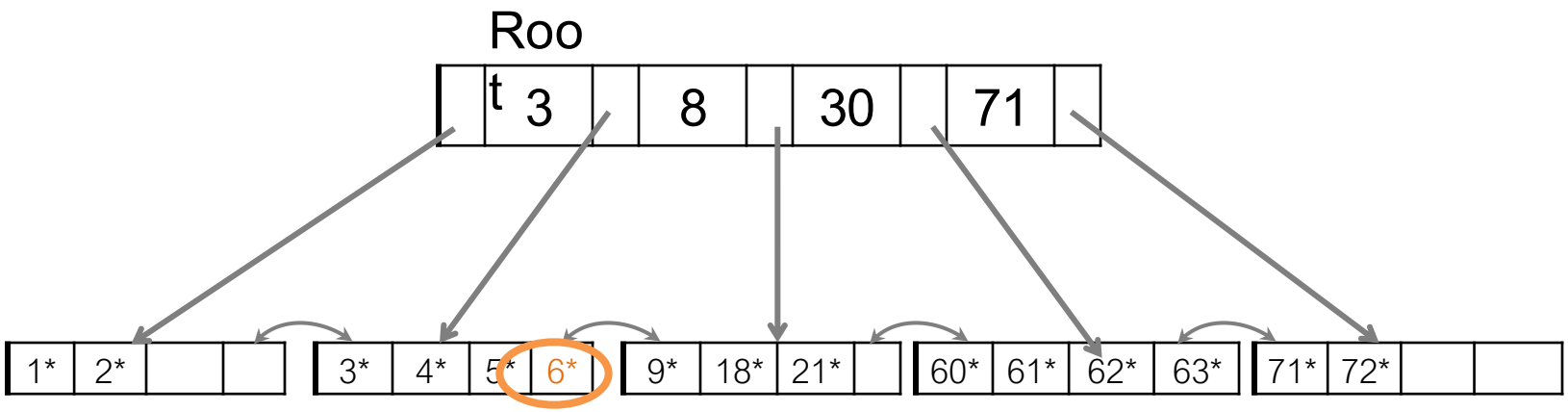
# Exercise

- Ins 6, ins 7, del 21, del 4 (in this order)
- Do not consider re-distribution for insertions (but redistribute for deletions)



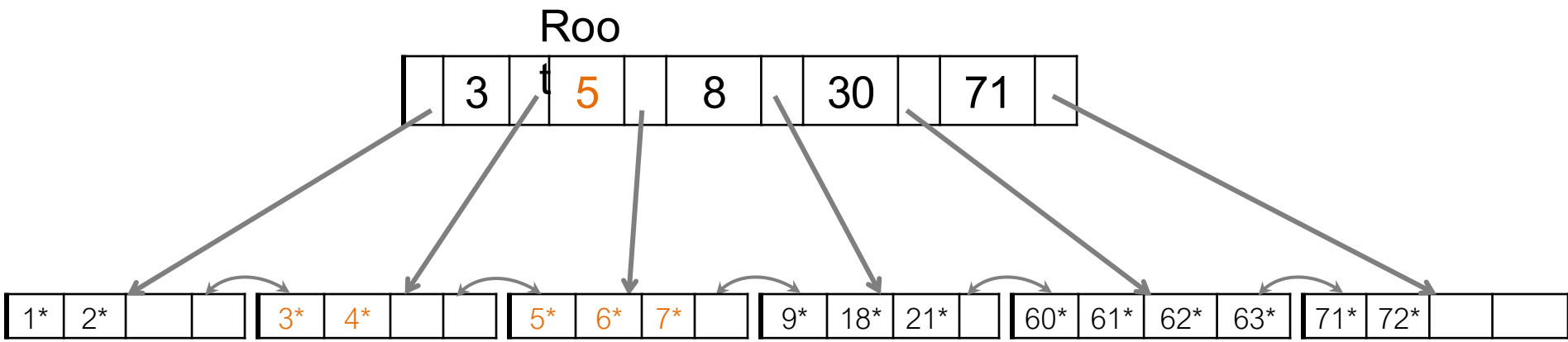
# Answer

Inserting 6 is simple



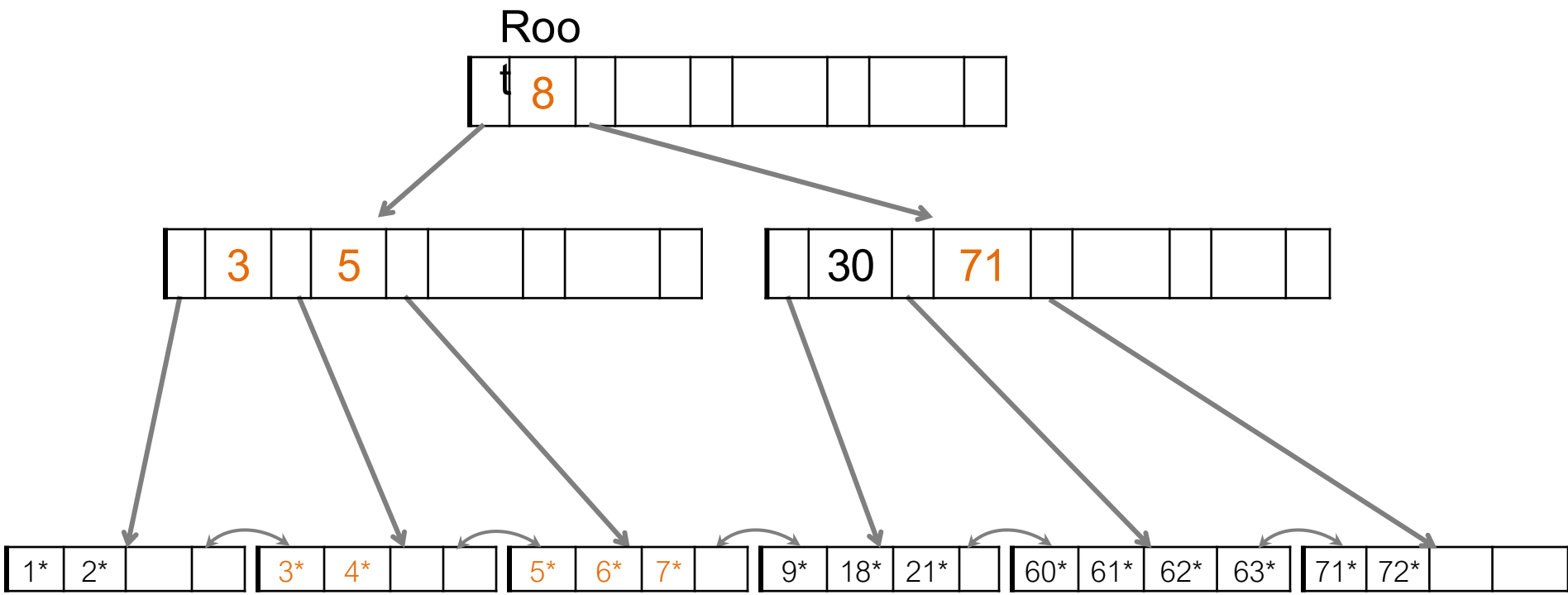
# Answer

- Inserting 7 causes a split
- We need to split its parent (here, root)



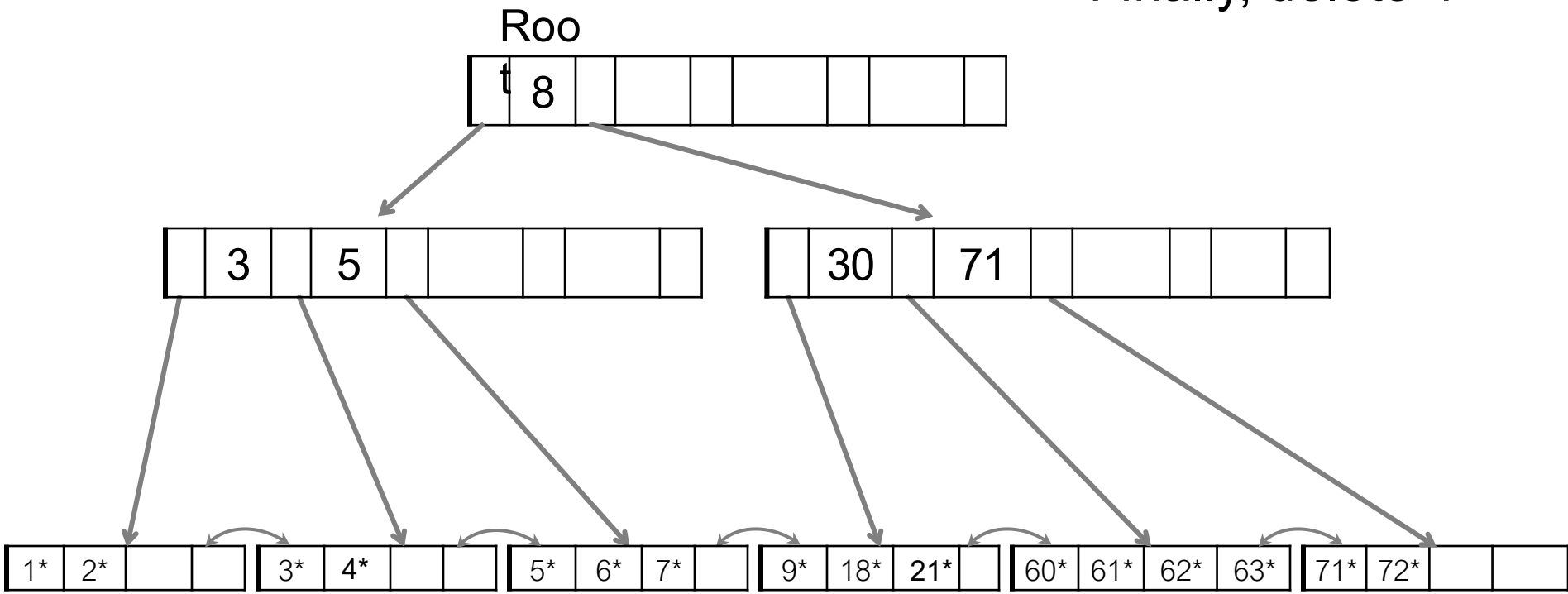
# Answer

Now we have a proper B+ tree again



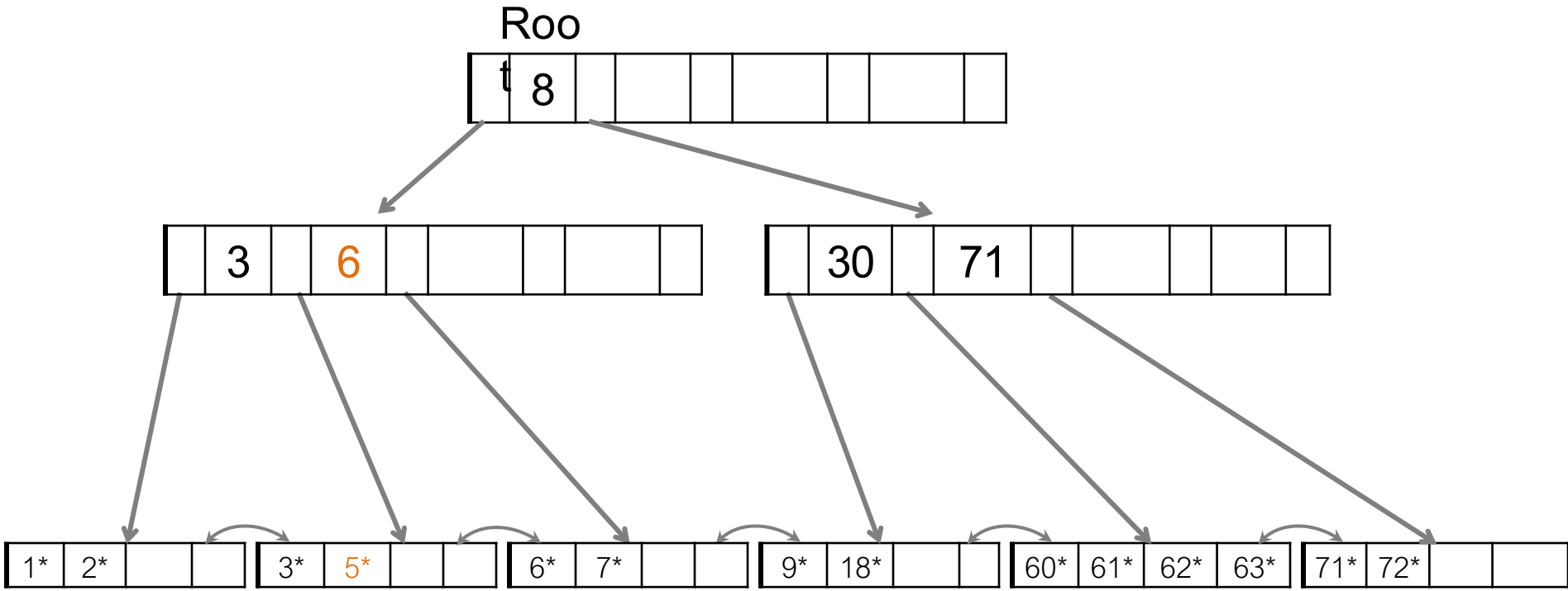
# Answer

- Delete 21 (easy)
- Finally, delete 4



# Answer

Now we need to borrow from sibling



# Suggested Exercises + Readings

Suggested Exercises: 10.1, 10.5, 10.7

Suggested readings for the lecture on hash-based index:

Read the entire chapter 11 (hash index)

# Next…